

推荐PM手把手入门第10讲：精排逻辑，从 LR、FM 到深度学习，PM 到底需要理解到什么程度？

👉 大家好，我是  大厂策略产品搬砖狗。

这是「推荐PM手把手入门专栏」的第十篇。上一讲我们聊了特征工程，讲的是怎么给模型"喂饭"。这一讲我们来聊聊精排模型本身——也就是这个"吃饭的人"到底是怎么工作的。

一、先说结论：PM 不需要推公式，但必须理解"为什么这样设计"

我先说我的观点：PM 不需要懂反向传播怎么求导，不需要手写梯度下降，但你必须理解每一代模型解决了什么问题，以及它为什么要这么设计。



PM对精排模型的理解层次

为什么？

因为当你在跟算法同学 pk 方案的时候，你得知道：

- 为什么我们不能直接用 LR 做精排？

- 为什么要上 FM，FM 解决了什么问题？
- 深度模型到底贵在哪？值不值得上？
- 我提的这个新特征，现有模型能不能"吃得下"？

这些问题，如果你只停留在"听说深度学习很厉害"的层面，是没法跟算法同学对话的。

所以今天这篇，我会用 **PM 视角** 把精排模型的演进逻辑讲清楚——不推公式，但会告诉你每一步为什么要这么走。

二、LR：最简单的起点，但也是"天花板"最明显的模型

LR 是什么？

LR（Logistic Regression，逻辑回归）是最经典的二分类模型，用在推荐里就是预测"用户会不会点击这个内容"。

它的核心逻辑非常简单：

1. 把所有特征（用户特征、内容特征、场景特征）加权求和
2. 通过一个 sigmoid 函数，把这个分数压缩到 0~1 之间
3. 这个数字就是"点击概率"

用一个更直观的方式理解：LR 其实就是在做"打分表"。

举个例子：

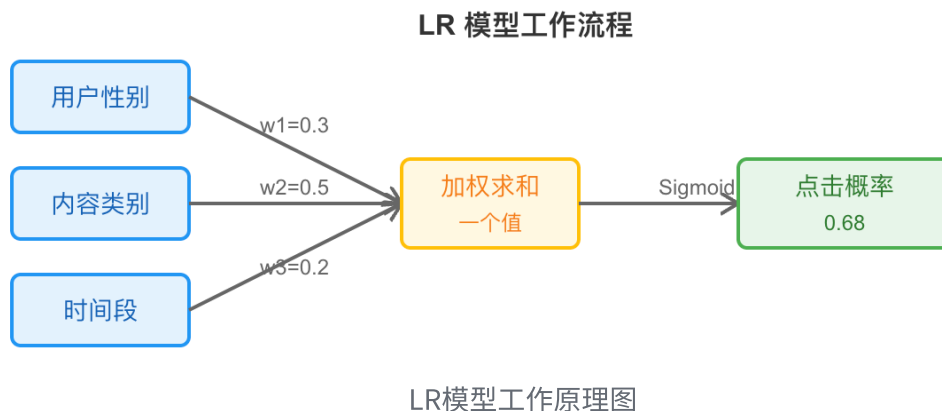
- 用户是女性：+0.3 分
- 内容是美妆类：+0.5 分
- 当前时间是晚上：+0.2 分
- 最后加起来，过一个函数，得到点击率 = 0.68

LR 的优点

- **简单、可解释**：每个特征的权重一目了然，出了问题容易 debug
- **训练快、推理快**：线性模型，计算量小

- **工程成本低**：很多早期推荐系统都是从 LR 起步的

LR 的工作原理



LR 的致命缺陷：不会"组合拳"

但 LR 有个巨大的问题：它假设所有特征是独立的。

什么意思？

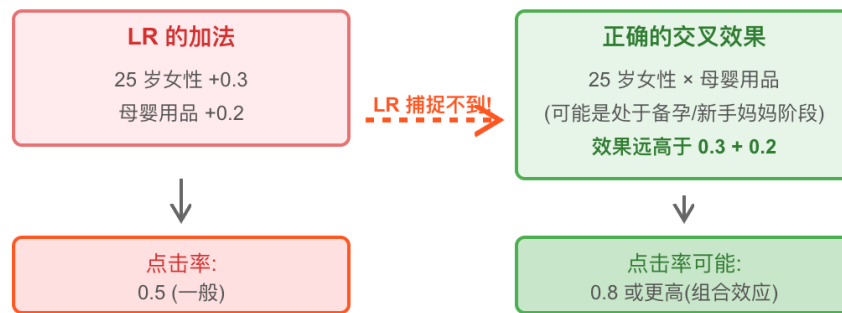
举个例子：

- 用户是"25 岁女性" → +0.3 分
- 内容是"母婴用品" → +0.2 分

但实际上，"25 岁女性 × 母婴用品"这个组合的点击率，可能远高于两个特征单独加起来的效应（因为 25 岁女性可能正处于备孕/新手妈妈阶段）。

这种"特征交叉"的价值，LR 是捕捉不到的。

LR 的缺陷:无法捕捉特征交叉



LR 只能的加法,无法捕捉特征交叉的组合价值

LR缺陷示意图

除非你手动构造一个"25 岁女性 & 母婴用品"的组合特征——但这样做的问题是：

- 特征组合爆炸 (n 个特征, 组合数是 n^2)
- 而且你根本不知道哪些组合有用, 哪些没用

所以 LR 的天花板, 就卡在了"无法自动学习特征交叉"这一步。

三、FM：让模型自己学会"组合拳"

FM 解决了什么问题？

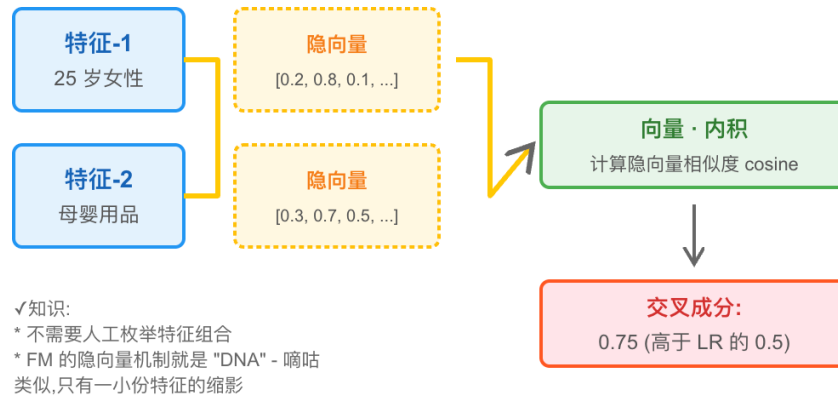
FM (Factorization Machine, 因子分解机) 的核心改进, 就是让模型自动学习二阶特征交叉。

还是刚才的例子：

- LR: 特征 A + 特征 B = 加法
- FM: 特征 A × 特征 B = 乘法 (交叉)

FM 的设计思路是：给每个特征学一个"隐向量"（可以理解为特征的"DNA"），然后通过向量内积来计算两个特征之间的交互强度。

FM 的特征交叉机制



FM 通过隐向量转化为个特征的 DNA,自动学习二阶交叉

FM工作原理图

用一个类比：

- 你可以把每个特征想象成一个人
- 每个人有一组"兴趣标签"（隐向量）
- 两个人的"化学反应"（交叉效果），就是他们兴趣标签的相似度

FM 的优势

- **自动学习交叉：**不需要人工枚举特征组合
- **泛化能力强：**即使某个特征组合在训练数据里很少见，FM 也能通过隐向量推断出它的效果
- **稀疏场景友好：**推荐场景里特征通常很稀疏（比如用户 ID、物品 ID 都是 one-hot），FM 的隐向量机制能有效处理

FM 的局限：只能做二阶交叉

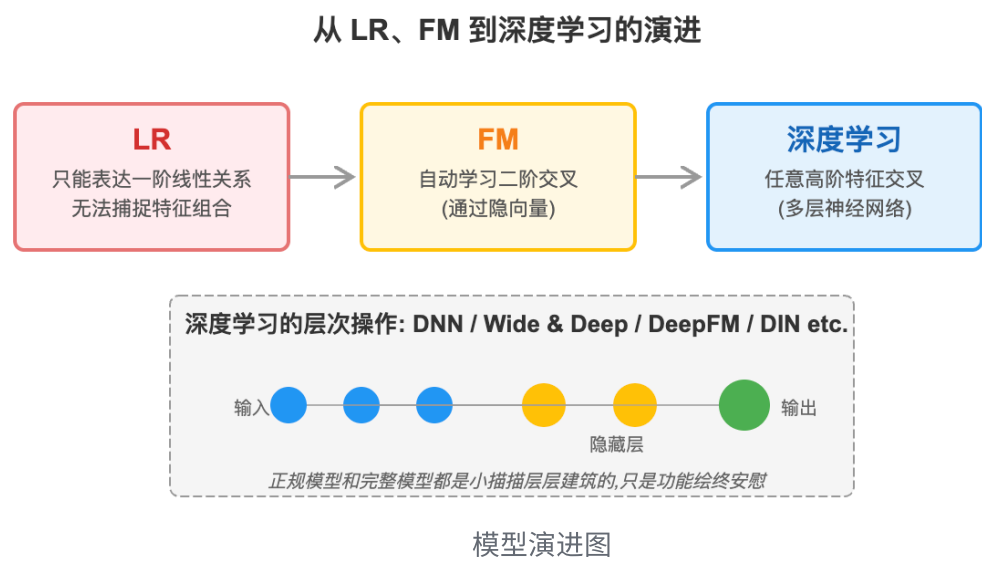
FM 解决了 LR 的痛点，但它也有自己的天花板：**FM 只能自动学习二阶交叉**（ $A \times B$ ），更高阶的交叉（ $A \times B \times C$ ）它做不了。

而且 FM 的交叉方式是"固定"的（向量内积），这种机制在某些复杂场景下表达能力还是不够。

四、深度学习：用"黑盒"换"上限"

为什么要上深度模型？

深度学习模型（比如 DNN、Wide & Deep、DeepFM、DIN 等）的核心优势是：它可以自动学习任意高阶的非线性特征交叉。



用一个更直观的说法：

- LR 是"一层逻辑"
- FM 是"两层逻辑"
- DNN 是"多层逻辑"，而且每一层都可以学到更抽象的模式

举个例子：

- 第一层：学到"用户性别 × 内容类目"
- 第二层：学到"（用户性别 × 内容类目） × 时间段"
- 第三层：学到"（（用户性别 × 内容类目） × 时间段） × 用户活跃度"

这种"层层递进"的学习能力，是 LR 和 FM 做不到的。

深度模型的代价

但深度模型也不是万能药，它有三个明显的代价：

1. 训练成本高：深度模型参数量大，训练时间长，需要更多的算力和数据。如果你的业务体量还没到一定规模，上深度模型的性价比可能不高。
2. 推理成本高：精排是在线服务，每次请求都要实时打分。深度模型的推理耗时比 LR/FM 高很多，这直接影响用户体验。所以很多团队会做"模型蒸馏"或者"分阶段精排"（粗排用轻模型，精排用重模型）。

3. 可解释性差：深度模型是"黑盒"，你很难知道它为什么给某个内容打了高分。这在 badcase 分析、策略调优的时候会很头疼。

五、PM 到底需要理解到什么程度？

说了这么多，回到最核心的问题：**PM 到底需要理解到什么程度？**

我的答案是：

1. 你要知道"为什么选这个模型"

当算法同学说"我们准备上 DeepFM"的时候，你要能问出：

- 为什么不用 FM？是因为二阶交叉不够吗？
- DeepFM 比 FM 的收益预期是多少？
- 训练和推理成本增加多少？值不值得？

2. 你要知道"这个模型的边界在哪"

举个例子：

- 如果你想做"用户长期兴趣建模"，你得知道普通 DNN 记不住历史，需要上 RNN/LSTM/Transformer 这类序列模型
- 如果你想做"用户实时行为建模"，你得知道 DIN（Deep Interest Network）是怎么通过 Attention 机制捕捉"当下兴趣"的

3. 你要知道"模型能不能吃得下你的特征"

这是最实际的一点。

当你提出一个新特征（比如"用户过去 30 天的点击序列"），你要能判断：

- 现有模型能不能处理序列特征？
- 如果不能，需要怎么改造？
- 改造的成本和收益是否匹配？

六、一个完整的决策链路：PM 怎么参与模型选型？

我举个真实场景的例子。

假设你在做一个内容推荐产品，现在算法同学说"我们的 LR 模型效果遇到瓶颈了，想升级"。

作为 PM，你应该怎么参与这个决策？

Step 1：先看数据和问题

你要先问：

- 瓶颈具体在哪？是整体 CTR 不涨了，还是某个细分场景效果差？
- 我们的特征工程做得怎么样？是不是还有优化空间？

很多时候，问题不在模型，而在特征。 如果特征本身就很弱，换再好的模型也没用。

Step 2：评估模型升级的必要性

如果确认是模型的问题，你要跟算法同学一起评估：

- 我们的业务场景，是不是真的需要高阶交叉？（比如电商场景的"品牌 × 价格 × 用户消费能力"组合）
- 数据量够不够支撑深度模型？（深度模型需要更多数据才能发挥优势）

Step 3：权衡成本和收益

假设决定上 DeepFM，你要推动算法同学给出：

- 预期的效果提升（比如 CTR +5%）
- 训练成本增加（比如训练时间从 2 小时变成 8 小时）
- 推理成本增加（比如 RT 从 20ms 变成 50ms）

然后你要做 ROI 判断：这个提升值不值得这个成本？

Step 4：灰度验证和回滚预案

模型上线不是一蹴而就的，你要推动：

- 小流量灰度（比如 5% 流量）
- 设置监控指标（CTR、RT、资源消耗）
- 准备回滚方案（如果效果不达预期或者稳定性有问题，随时切回旧模型）

七、写在最后：PM 的价值不是"懂算法"，而是"懂权衡"

很多 PM 会焦虑"我不懂算法，是不是做不好推荐产品"。

但我想说：**PM 的价值不在于比算法同学更懂算法，而在于你能站在产品和业务的视角，做出正确的权衡。**

你要懂的是：

- 这个模型解决了什么问题？
- 这个问题对业务有多重要？
- 解决这个问题的成本是多少？
- 有没有更轻量的方案？

这些判断，需要你对模型有"概念级"的理解，但不需要你去推公式、写代码。

所以，回到今天的主题：**从 LR、FM 到深度学习，PM 到底需要理解到什么程度？**

我的答案是：**理解每一代模型的设计动机、优势和局限，能跟算法同学对话，能做出合理的权衡和决策。**

仅此而已，也恰到好处。